



Data Structures

数据结构

boshi

长沙市一中

2020 年 8 月 1 日

1 简介

- 我是谁
- 数据结构是什么

2 正文

- 并查集
 - Anton and Tree (CF734E)
 - 染色问题
 - 方格染色 (luoguP3631)
 - 总结一下
- 队列
 - 琪露诺 (luoguP1725)
 - Bank notes (BZOJ1531)
 - 玩具装箱 (luoguP3195)
 - 挖油 (BZOJ2448)
 - 稻草人 (BZOJ4237)
 - 总结一下
- 栈
 - 双栈排序 (luoguP1155)
 - Harbingers (BZOJ1767)
 - 去月球 (UOJ327)

- 楼房重建 (luoguP4198)
- 一个奇怪的题目
- String Set Queries (CF710F)
- 玄学 (UOJ46)
- String Set Queries 加强版
- 总结一下
- 线段树
 - DZY Loves Fibonacci Numbers (CF446C)
 - Tree Generator TM (CF1149C)
 - HH 的项链 (luoguP1972)
 - 神牛的养成计划 (BZOJ4212)
 - 火星商店问题 (luoguP4585)
 - 树据结构 (51nod1462)
 - 总结一下
- 其他数据结构
 - 罗马游戏 (BZOJ1455)
 - Lomsat gelral (CF600E)
 - 总结一下

3 题目推荐

1 简介

我是谁

数据结构是什么

2 正文

并查集

- Anton and Tree (CF734E)
- 染色问题
- 方格染色 (luoguP3631)
- 总结一下

队列

- 琪露诺 (luoguP1725)
- Bank notes (BZOJ1531)
- 玩具装箱 (luoguP3195)
- 挖油 (BZOJ2448)
- 稻草人 (BZOJ4237)
- 总结一下

栈

- 双栈排序 (luoguP1155)
- Harbingers (BZOJ1767)
- 去月球 (UOJ327)

- 楼房重建 (luoguP4198)
- 一个奇怪的题目
- String Set Queries (CF710F)
- 玄学 (UOJ46)
- String Set Queries 加强版
- 总结一下

线段树

- DZY Loves Fibonacci Numbers (CF446C)
- Tree Generator TM (CF1149C)
- HH 的项链 (luoguP1972)
- 神牛的养成计划 (BZOJ4212)
- 火星商店问题 (luoguP4585)
- 树据结构 (51nod1462)
- 总结一下

其他数据结构

- 罗马游戏 (BZOJ1455)
- Lomsat gelral (CF600E)
- 总结一下

3 题目推荐

Outline

简介

我是谁

数据结构是什么

正文

并查集

队列

栈

线段树

其他数据结构

题目推荐

1 简介

- 我是谁
- 数据结构是什么

2 正文

- 并查集
 - Anton and Tree (CF734E)
 - 染色问题
 - 方格染色 (luoguP3631)
 - 总结一下
- 队列
 - 琪露诺 (luoguP1725)
 - Bank notes (BZOJ1531)
 - 玩具装箱 (luoguP3195)
 - 挖油 (BZOJ2448)
 - 稻草人 (BZOJ4237)
 - 总结一下
- 栈
 - 双栈排序 (luoguP1155)
 - Harbingers (BZOJ1767)
 - 去月球 (UOJ327)

- 楼房重建 (luoguP4198)
- 一个奇怪的题目
- String Set Queries (CF710F)
- 玄学 (UOJ46)
- String Set Queries 加强版
- 总结一下
- 线段树
 - DZY Loves Fibonacci Numbers (CF446C)
 - Tree Generator TM (CF1149C)
 - HH 的项链 (luoguP1972)
 - 神牛的养成计划 (BZOJ4212)
 - 火星商店问题 (luoguP4585)
 - 树据结构 (51nod1462)
 - 总结一下
- 其他数据结构
 - 罗马游戏 (BZOJ1455)
 - Lomsat gelral (CF600E)
 - 总结一下

3 题目推荐

在 NOIP 提高组中，可能考察到的数据结构大致有：

- 栈
- 队列
- 堆
- 并查集
- 树状数组
- stl 中的 set、map、vector、hashmap 等
- 线段树
- 平衡树
- ST 表
- 分块结构

学习数据结构分为两个重要阶段。

- 1 学会打模板 (模板题)
- 2 学会灵活应用

相信大家已经走完了第一个阶段。那我们就直接从灵活运用数据结构的题目开始。

1 简介

- 我是谁
- 数据结构是什么

2 正文

并查集

- Anton and Tree (CF734E)
- 染色问题
- 方格染色 (luoguP3631)
- 总结一下

队列

- 琪露诺 (luoguP1725)
- Bank notes (BZOJ1531)
- 玩具装箱 (luoguP3195)
- 挖油 (BZOJ2448)
- 稻草人 (BZOJ4237)
- 总结一下

栈

- 双栈排序 (luoguP1155)
- Harbingers (BZOJ1767)
- 去月球 (UOJ327)

- 楼房重建 (luoguP4198)
- 一个奇怪的题目
- String Set Queries (CF710F)
- 玄学 (UOJ46)
- String Set Queries 加强版
- 总结一下

线段树

- DZY Loves Fibonacci Numbers (CF446C)
- Tree Generator TM (CF1149C)
- HH 的项链 (luoguP1972)
- 神牛的养成计划 (BZOJ4212)
- 火星商店问题 (luoguP4585)
- 树据结构 (51nod1462)
- 总结一下

其他数据结构

- 罗马游戏 (BZOJ1455)
- Lomsat gelral (CF600E)
- 总结一下

3 题目推荐

Anton and Tree (CF734E)

给定一棵树，点为白色或者黑色。

每次可以将一个同色连通块改变颜色（黑变白，白变黑）

问最少多少次可以将整棵树改为同一种颜色。树大小

$n \leq 2 \times 10^5$ 。

- 显然一个同色连通块可以合并为一个点。

- 显然一个同色连通块可以合并为一个点。
- 我们可以从缩点之后的树的直径开始，每次向外扩展一层。

- 显然一个同色连通块可以合并为一个点。
- 我们可以从缩点之后的树的直径开始，每次向外扩展一层。
- 在 $\lfloor \frac{len}{2} \rfloor$ 次操作后，整棵树就变成同色的了。其中 len 为直径长度。

染色问题

给一串珠子染色，每次给第 $[l_i, r_i]$ 个珠子染上第 k_i 种颜色。
最终每个珠子的颜色为它被染到的序号最大的颜色。
求每个珠子最后颜色。支持离线。

- 我们可以将操作离线下来，然后按照编号从大到小的顺序给珠子染色。

- 我们可以将操作离线下来，然后按照编号从大到小的顺序给珠子染色。
- 如果一个珠子已经被染色过，则跳过该珠子。

- 我们可以将操作离线下来，然后按照编号从大到小的顺序给珠子染色。
- 如果一个珠子已经被染色过，则跳过该珠子。
- 我们可以用并查集来维护每个珠子右边第一个（包括自己）还没有被染色的珠子的编号：并查集的根即为右（包括自己）边第一个没有被染色的珠子。

- 我们可以将操作离线下来，然后按照编号从大到小的顺序给珠子染色。
- 如果一个珠子已经被染色过，则跳过该珠子。
- 我们可以用并查集来维护每个珠子右边第一个（包括自己）还没有被染色的珠子的编号：并查集的根即为右（包括自己）边第一个没有被染色的珠子。
- 一开始，所有珠子各自的并查集都只有一个点。

- 我们可以将操作离线下来，然后按照编号从大到小的顺序给珠子染色。
- 如果一个珠子已经被染色过，则跳过该珠子。
- 我们可以用并查集来维护每个珠子右边第一个（包括自己）还没有被染色的珠子的编号：并查集的根即为右边第一个没有被染色的珠子。
- 一开始，所有珠子各自的并查集都只有一个点。
- 当一个珠子被染色后，将它所在的并查集的根设为它右边的珠子所在的并查集。

- 我们可以将操作离线下来，然后按照编号从大到小的顺序给珠子染色。
- 如果一个珠子已经被染色过，则跳过该珠子。
- 我们可以用并查集来维护每个珠子右边第一个（包括自己）还没有被染色的珠子的编号：并查集的根即为右（包括自己）边第一个没有被染色的珠子。
- 一开始，所有珠子各自的并查集都只有一个点。
- 当一个珠子被染色后，将它所在的并查集的根设为它右边的珠子所在的并查集。
- 这样我们就可以均摊 $O(1)$ 获得某个珠子下一个没有被染色的珠子。每次染色从 l_i 开始，将 $[l_i, r_i]$ 之间所有没有被染色的珠子染色即可。

- 我们可以将操作离线下来，然后按照编号从大到小的顺序给珠子染色。
- 如果一个珠子已经被染色过，则跳过该珠子。
- 我们可以用并查集来维护每个珠子右边第一个（包括自己）还没有被染色的珠子的编号：并查集的根即为右边第一个没有被染色的珠子。
- 一开始，所有珠子各自的并查集都只有一个点。
- 当一个珠子被染色后，将它所在的并查集的根设为它右边的珠子所在的并查集。
- 这样我们就可以均摊 $O(1)$ 获得某个珠子下一个没有被染色的珠子。每次染色从 l_i 开始，将 $[l_i, r_i]$ 之间所有没有被染色的珠子染色即可。
- 这样做的总时间复杂度就是 $O(n)$ 的。

方格染色 (luoguP3631)

将一个 $n \times m (n, m \leq 10^5)$ 的棋盘染色，每个格子为红色或蓝色。

其中已经有 $k (k \leq 10^5)$ 个格子被染成了红色或者蓝色。要求最终的染色方案满足，每个 2×2 的子棋盘中都有奇数个红色格子。

请问对剩下的格子有多少种可行的染色方法。

- 设红色为 1 , 蓝色为 0 。

- 设红色为 1 , 蓝色为 0 。
- 那么任意一个 2×2 的子矩阵的异或值都为 1 。

- 设红色为 1 , 蓝色为 0 。
- 那么任意一个 2×2 的子矩阵的异或值都为 1 。
- 我们可以选取若干个特定的子矩阵异或起来, 得到形式如下的方程:

$$A(1,1) \oplus A(1,y) \oplus A(x,1) \oplus A(x,y) = 0/1$$

- 设红色为 1 , 蓝色为 0 。
- 那么任意一个 2×2 的子矩阵的异或值都为 1 。
- 我们可以选取若干个特定的子矩阵异或起来, 得到形式如下的方程:

$$A(1,1) \oplus A(1,y) \oplus A(x,1) \oplus A(x,y) = 0/1$$

- 根据给定的 k 个 $A(x,y) = 0/1$ 的条件, 我们可以列出 k 个方程。

- 设红色为 1 , 蓝色为 0 。
- 那么任意一个 2×2 的子矩阵的异或值都为 1 。
- 我们可以选取若干个特定的子矩阵异或起来, 得到形式如下的方程:

$$A(1,1) \oplus A(1,y) \oplus A(x,1) \oplus A(x,y) = 0/1$$

- 根据给定的 k 个 $A(x,y) = 0/1$ 的条件, 我们可以列出 k 个方程。
- 现在的任务就是确定方程组的解数。

- 设红色为 1 , 蓝色为 0 。
- 那么任意一个 2×2 的子矩阵的异或值都为 1 。
- 我们可以选取若干个特定的子矩阵异或起来, 得到形式如下的方程:

$$A(1,1) \oplus A(1,y) \oplus A(x,1) \oplus A(x,y) = 0/1$$

- 根据给定的 k 个 $A(x,y) = 0/1$ 的条件, 我们可以列出 k 个方程。
- 现在的任务就是确定方程组的解数。
- 如果我们枚举 $A(1,1)$ 的值, 方程组的形式会变成:

$$A(x_i,1) \oplus A(y_i,1) = 0/1$$

- 设红色为 1 , 蓝色为 0 。
- 那么任意一个 2×2 的子矩阵的异或值都为 1 。
- 我们可以选取若干个特定的子矩阵异或起来, 得到形式如下的方程:

$$A(1,1) \oplus A(1,y) \oplus A(x,1) \oplus A(x,y) = 0/1$$

- 根据给定的 k 个 $A(x,y) = 0/1$ 的条件, 我们可以列出 k 个方程。
- 现在的任务就是确定方程组的解数。
- 如果我们枚举 $A(1,1)$ 的值, 方程组的形式会变成:

$$A(x_i,1) \oplus A(y_i,1) = 0/1$$

- 将每个变量看成一个点, 一个方程看成一条边。那么一个连通块对应的方程组一定只有 0 或 2 组解。

- 设红色为 1 , 蓝色为 0 。
- 那么任意一个 2×2 的子矩阵的异或值都为 1 。
- 我们可以选取若干个特定的子矩阵异或起来, 得到形式如下的方程:

$$A(1,1) \oplus A(1,y) \oplus A(x,1) \oplus A(x,y) = 0/1$$

- 根据给定的 k 个 $A(x,y) = 0/1$ 的条件, 我们可以列出 k 个方程。
- 现在的任务就是确定方程组的解数。
- 如果我们枚举 $A(1,1)$ 的值, 方程组的形式会变成:

$$A(x_i,1) \oplus A(y_i,1) = 0/1$$

- 将每个变量看成一个点, 一个方程看成一条边。那么一个连通块对应的方程组一定只有 0 或 2 组解。
- 而整个问题的解数就是每个连通块的解数的积。

总结一下:

总结一下：

- 并查集在 OI 中有着广泛的应用。

总结一下：

- 并查集在 OI 中有着广泛的应用。
- 最简单的维护连通性的问题需要用到 (Anton and Tree)。

总结一下：

- 并查集在 OI 中有着广泛的应用。
- 最简单的维护连通性的问题需要用到 (Anton and Tree)。
- 维护序列的后继可以使用并查集优化到 $O(n)$ (染色问题)。

总结一下：

- 并查集在 OI 中有着广泛的应用。
- 最简单的维护连通性的问题需要用到 (Anton and Tree)。
- 维护序列的后继可以使用并查集优化到 $O(n)$ (染色问题)。
- 带权并查集可以用于询问连通块内点之间的“距离” (方格染色)。

1 简介

- 我是谁
- 数据结构是什么

2 正文

- 并查集
 - Anton and Tree (CF734E)
 - 染色问题
 - 方格染色 (luoguP3631)
 - 总结一下
- 队列
 - 琪露诺 (luoguP1725)
 - Bank notes (BZOJ1531)
 - 玩具装箱 (luoguP3195)
 - 挖油 (BZOJ2448)
 - 稻草人 (BZOJ4237)
 - 总结一下
- 栈
 - 双栈排序 (luoguP1155)
 - Harbingers (BZOJ1767)
 - 去月球 (UOJ327)

- 楼房重建 (luoguP4198)
- 一个奇怪的题目
- String Set Queries (CF710F)
- 玄学 (UOJ46)
- String Set Queries 加强版
- 总结一下
- 线段树
 - DZY Loves Fibonacci Numbers (CF446C)
 - Tree Generator TM (CF1149C)
 - HH 的项链 (luoguP1972)
 - 神牛的养成计划 (BZOJ4212)
 - 火星商店问题 (luoguP4585)
 - 树据结构 (51nod1462)
 - 总结一下
- 其他数据结构
 - 罗马游戏 (BZOJ1455)
 - Lomsat gelral (CF600E)
 - 总结一下

3 题目推荐

琪露诺 (luoguP1725)

给定一条数轴，需要从 0 跳到坐标大于 $n(n \leq 2 \times 10^5)$ 的地方。中间每一步可以跳的步长为 $[l, r]$ 之间的整数。每跳到一个点，获得该点对应的分数。给定每个点的分数，求最高得分。

- 设 $f[i]$ 为跳到 i 点的过程中可以收集的最高分数。则很容易写出状态转移方程：

$$f[i] = \max\{f[j] \mid i - j \in [l, r]\} + a[i]$$

- 设 $f[i]$ 为跳到 i 点的过程中可以收集的最高分数。则很容易写出状态转移方程：

$$f[i] = \max\{f[j] \mid i - j \in [l, r]\} + a[i]$$

- 注意到 i 每增加一，可以转移的 $f[j]$ 的集合基本上没有改变。

- 设 $f[i]$ 为跳到 i 点的过程中可以收集的最高分数。则很容易写出状态转移方程：

$$f[i] = \max\{f[j] \mid i - j \in [l, r]\} + a[i]$$

- 注意到 i 每增加一，可以转移的 $f[j]$ 的集合基本上没有改变。
- 因此根据题目的性质我们可以用单调队列维护可以转移的 $f[j]$ 的背包可以产生的最大价值。

- 设 $f[i]$ 为跳到 i 点的过程中可以收集的最高分数。则很容易写出状态转移方程：

$$f[i] = \max\{f[j] \mid i - j \in [l, r]\} + a[i]$$

- 注意到 i 每增加一，可以转移的 $f[j]$ 的集合基本上没有改变。
- 因此根据题目的性质我们可以用单调队列维护可以转移的 $f[j]$ 的背包可以产生的最大价值。
- 算法的复杂度为 $O(n)$ 。

Bank notes (BZOJ1531)

给定一些硬币的面值 s 和它们的数量 t ，求凑成给定面值的最少硬币数为多少。

硬币种数 $n(n \leq 200)$ ，需要凑出的面值为 $k(k \leq 200000)$ 。

Bank notes (BZOJ1531)

给定一些硬币的面值 s 和它们的数量 t ，求凑成给定面值的最少硬币数为多少。

硬币种数 $n(n \leq 200)$ ，需要凑出的面值为 $k(k \leq 200000)$ 。

- 令 $f[i]$ 为凑出面额 i 的最小硬币数。

Bank notes (BZOJ1531)

给定一些硬币的面值 s 和它们的数量 t ，求凑成给定面值的最少硬币数为多少。

硬币种数 $n(n \leq 200)$ ，需要凑出的面值为 $k(k \leq 200000)$ 。

- 令 $f[i]$ 为凑出面额 i 的最小硬币数。
- 依次枚举每个面值的硬币，然后转移 $f[i]$ ：

$$f[i] = \min\{f[i - ks] + k \mid k \leq t\}$$

Bank notes (BZOJ1531)

给定一些硬币的面值 s 和它们的数量 t ，求凑成给定面值的最少硬币数为多少。

硬币种数 $n(n \leq 200)$ ，需要凑出的面值为 $k(k \leq 200000)$ 。

- 令 $f[i]$ 为凑出面额 i 的最小硬币数。
- 依次枚举每个面值的硬币，然后转移 $f[i]$ ：

$$f[i] = \min\{f[i - ks] + k \mid k \leq t\}$$

- 注意到只有对 s 取模的余数相同的状态直接存在转移关系，我们不妨将这些状态提出来，分别转移：

$$f[i] = \min\{f[i - k] + k \mid k \leq t\} = \min\{f[j] - j \mid i - j \geq t\} + i$$

Bank notes (BZOJ1531)

给定一些硬币的面值 s 和它们的数量 t ，求凑成给定面值的最少硬币数为多少。

硬币种数 $n(n \leq 200)$ ，需要凑出的面值为 $k(k \leq 200000)$ 。

- 令 $f[i]$ 为凑出面额 i 的最小硬币数。
- 依次枚举每个面值的硬币，然后转移 $f[i]$ ：

$$f[i] = \min\{f[i - ks] + k \mid k \leq t\}$$

- 注意到只有对 s 取模的余数相同的状态直接存在转移关系，我们不妨将这些状态提出来，分别转移：

$$f[i] = \min\{f[i - k] + k \mid k \leq t\} = \min\{f[j] - j \mid i - j \geq t\} + i$$

- 此时与 j 有关的项已经完全分离出来，只需要用单调队列就可以快速转移了。复杂度 $O(nk)$ 。

玩具装箱 (luoguP3195)

给定一个序列 $\{a_i\}$ ，你需要将这个序列分割为若干段，设每一段的和为 s_i ，求

$$\sum_i (s_i - L)^2$$

的最小值。

- 我们同样先写出状态转移方程：

$$f[i] = \min\{f[j] + (S_i - S_j - L)^2 | j < i\}$$

- 我们同样先写出状态转移方程：

$$f[i] = \min\{f[j] + (S_i - S_j - L)^2 | j < i\}$$

- 仿照上一题，分离变量：

$$f[i] = \min\{(f[j] + S_j^2 + 2LS_j) - 2S_iS_j | j < i\} + S_i^2 - 2LS_i + L^2$$

- 我们同样先写出状态转移方程：

$$f[i] = \min\{f[j] + (S_i - S_j - L)^2 | j < i\}$$

- 仿照上一题，分离变量：

$$f[i] = \min\{(f[j] + S_j^2 + 2LS_j) - 2S_iS_j | j < i\} + S_i^2 - 2LS_i + L^2$$

- 不难发现，除了 $-2S_iS_j$ 这一项之外，与 i 和与 j 有关的项都被分开了。

- 我们同样先写出状态转移方程：

$$f[i] = \min\{f[j] + (S_i - S_j - L)^2 | j < i\}$$

- 仿照上一题，分离变量：

$$f[i] = \min\{(f[j] + S_j^2 + 2LS_j) - 2S_iS_j | j < i\} + S_i^2 - 2LS_i + L^2$$

- 不难发现，除了 $-2S_iS_j$ 这一项之外，与 i 和与 j 有关的项都被分开了。
- 此时，我们对问题进行“升维打击”：

$$f[i] = \min\{(-2S_j, f[j] + S_j^2 + 2LS_j) \cdot (S_i, 1) | j < i\} + W_i$$

- 我们同样先写出状态转移方程：

$$f[i] = \min\{f[j] + (S_i - S_j - L)^2 | j < i\}$$

- 仿照上一题，分离变量：

$$f[i] = \min\{(f[j] + S_j^2 + 2LS_j) - 2S_iS_j | j < i\} + S_i^2 - 2LS_i + L^2$$

- 不难发现，除了 $-2S_iS_j$ 这一项之外，与 i 和与 j 有关的项都被分开了。
- 此时，我们对问题进行“升维打击”：

$$f[i] = \min\{(-2S_j, f[j] + S_j^2 + 2LS_j) \cdot (S_i, 1) | j < i\} + W_i$$

- 把前者想象成二维平面中的点，后者想象成一个向量，一切豁然开朗。

- 我们同样先写出状态转移方程：

$$f[i] = \min\{f[j] + (S_i - S_j - L)^2 | j < i\}$$

- 仿照上一题，分离变量：

$$f[i] = \min\{(f[j] + S_j^2 + 2LS_j) - 2S_iS_j | j < i\} + S_i^2 - 2LS_i + L^2$$

- 不难发现，除了 $-2S_iS_j$ 这一项之外，与 i 和与 j 有关的项都被分开了。
- 此时，我们对问题进行“升维打击”：

$$f[i] = \min\{(-2S_j, f[j] + S_j^2 + 2LS_j) \cdot (S_i, 1) | j < i\} + W_i$$

- 把前者想象成二维平面中的点，后者想象成一个向量，一切豁然开朗。
- 使用单调队列维护点集的凸包，可以做到 $O(n)$ 。

挖油 (BZOJ2448)

有一个未知的 01 序列，其中以中间某个地方为界，左侧全部都是 1，右侧全部都是 0（有可能全是 1 或者 0）。

每次可以询问下标为 i 的位是 1 还是 0，但是需要耗费代价 $A[i]$ 。

求：最少花费多少的代价就一定可以确定最靠右的 1 的位置。
序列长度 $n \leq 2000$ 。

挖油 (BZOJ2448)

有一个未知的 01 序列，其中以中间某个地方为界，左侧全部都是 1，右侧全部都是 0（有可能全是 1 或者 0）。

每次可以询问下标为 i 的位是 1 还是 0，但是需要耗费代价 $A[i]$ 。

求：最少花费多少的代价就一定可以确定最靠右的 1 的位置。
序列长度 $n \leq 2000$ 。

- 很显然，我们可以写出状态转移方程：

$$f[i][j] = \begin{cases} \min\{\max\{f[k+1][j], f[i][k-1]\} + f[k] \mid i \leq k \leq j\} & (i \leq j) \\ 0 & (i > j) \end{cases}$$

挖油 (BZOJ2448)

有一个未知的 01 序列，其中以中间某个地方为界，左侧全部都是 1，右侧全部都是 0（有可能全是 1 或者 0）。

每次可以询问下标为 i 的位是 1 还是 0，但是需要耗费代价 $A[i]$ 。

求：最少花费多少的代价就一定可以确定最靠右的 1 的位置。
序列长度 $n \leq 2000$ 。

- 很显然，我们可以写出状态转移方程：

$$f[i][j] = \begin{cases} \min\{\max\{f[k+1][j], f[i][k-1]\} + f[k] \mid i \leq k \leq j\} & (i \leq j) \\ 0 & (i > j) \end{cases}$$

- 直接转移是 $O(n^3)$ 的。

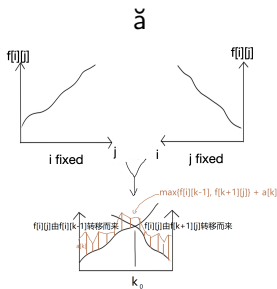
决策单调性

令 $g[i][j]$ 为使 $f[i][j]$ 取到最小值的 k 。则：

$$f[i][j] \leq f[i][j+1], f[i][j] \leq f[i-1][j]$$

$$g[i][j] \leq g[i][j+1], g[i][j] \geq g[i-1][j]$$

画个图：



图：状态转移方程的直观表示

- 假设图片中间那个交点是 k_0 , 则:

$$f[i][j] = \min\{\min_{k \leq k_0}\{f[i][k-1] + a[k]\}, \min_{k > k_0}\{f[k+1][j] + a[k]\}\}$$

- 假设图片中间那个交点是 k_0 ，则：

$$f[i][j] = \min\{\min_{k \leq k_0}\{f[i][k-1] + a[k]\}, \min_{k > k_0}\{f[k+1][j] + a[k]\}\}$$

- 我们可以倒序枚举 i ，顺序枚举 j ，然后根据决策单调性动态维护 k_0 的位置。

- 假设图片中间那个交点是 k_0 ，则：

$$f[i][j] = \min\{\min_{k \leq k_0}\{f[i][k-1] + a[k]\}, \min_{k > k_0}\{f[k+1][j] + a[k]\}\}$$

- 我们可以倒序枚举 i ，顺序枚举 j ，然后根据决策单调性动态维护 k_0 的位置。
- 同时，用单调队列维护 $f[i][k-1] + a[k]$ 和 $f[k+1] + a[k]$ 的区间最小值，就可以均摊 $O(1)$ 转移了。

- 假设图片中间那个交点是 k_0 ，则：

$$f[i][j] = \min\{\min_{k \leq k_0}\{f[i][k-1] + a[k]\}, \min_{k > k_0}\{f[k+1][j] + a[k]\}\}$$

- 我们可以倒序枚举 i ，顺序枚举 j ，然后根据决策单调性动态维护 k_0 的位置。
- 同时，用单调队列维护 $f[i][k-1] + a[k]$ 和 $f[k+1][j] + a[k]$ 的区间最小值，就可以均摊 $O(1)$ 转移了。
 - 右端点向右移动的时候，如果 k_0 也跟着移动，则将元素压入第一种单调队列。
 - 左端点向左移动的时候，如果 k_0 也跟着移动，则将元素压入第二种单调队列。

- 假设图片中间那个交点是 k_0 ，则：

$$f[i][j] = \min\{\min_{k \leq k_0}\{f[i][k-1] + a[k]\}, \min_{k > k_0}\{f[k+1][j] + a[k]\}\}$$

- 我们可以倒序枚举 i ，顺序枚举 j ，然后根据决策单调性动态维护 k_0 的位置。
- 同时，用单调队列维护 $f[i][k-1] + a[k]$ 和 $f[k+1][j] + a[k]$ 的区间最小值，就可以均摊 $O(1)$ 转移了。
 - 右端点向右移动的时候，如果 k_0 也跟着移动，则将元素压入第一种单调队列。
 - 左端点向左移动的时候，如果 k_0 也跟着移动，则将元素压入第二种单调队列。
- 这样，第一种单调队列维护的就是 $\min\{f[i][k-1] + a[k]\}$ ，第二种单调队列维护的则是 $\min\{f[k+1][j] + a[k]\}$ 。

- 假设图片中间那个交点是 k_0 ，则：

$$f[i][j] = \min\{\min_{k \leq k_0}\{f[i][k-1] + a[k]\}, \min_{k > k_0}\{f[k+1][j] + a[k]\}\}$$

- 我们可以倒序枚举 i ，顺序枚举 j ，然后根据决策单调性动态维护 k_0 的位置。
- 同时，用单调队列维护 $f[i][k-1] + a[k]$ 和 $f[k+1][j] + a[k]$ 的区间最小值，就可以均摊 $O(1)$ 转移了。
 - 右端点向右移动的时候，如果 k_0 也跟着移动，则将元素压入第一种单调队列。
 - 左端点向左移动的时候，如果 k_0 也跟着移动，则将元素压入第二种单调队列。
- 这样，第一种单调队列维护的就是 $\min\{f[i][k-1] + a[k]\}$ ，第二种单调队列维护的则是 $\min\{f[k+1][j] + a[k]\}$ 。
- 时间复杂度 $O(n^2)$ 。

稻草人 (BZOJ4237)

给定平面上一堆点，求有多少个逆序对确定的矩形中间没有其他点。

稻草人 (BZOJ4237)

给定平面上一堆点，求有多少个逆序对确定的矩形中间没有其他点。

- 首先，我们可以采用 CDQ 分治将问题划分为若干个更简单一点的问题：

稻草人 (BZOJ4237)

给定平面上一堆点，求有多少个逆序对确定的矩形中间没有其他点。

- 首先，我们可以采用 CDQ 分治将问题划分为若干个更简单一点的问题：

简化后的问题

给定第一象限内一个点集和第四象限内一个点集。求有多少逆序对跨越两个象限，且确定的矩形中间没有其他点。

稻草人 (BZOJ4237)

给定平面上一堆点，求有多少个逆序对确定的矩形中间没有其他点。

- 首先，我们可以采用 CDQ 分治将问题划分为若干个更简单一点的问题：

简化后的问题

给定第一象限内一个点集和第四象限内一个点集。求有多少逆序对跨越两个象限，且确定的矩形中间没有其他点。

- 按 x 逆序遍历每个点。对第一象限内的每个点都求简化后的问题的答案。

- 这个矩形以第一象限中的一个点 P_i 为左上角。

- 这个矩形以第一象限中的一个点 P_i 为左上角。
- 首先，这个矩形不能覆盖第一象限中第一个低于这个点的点 Q_i 。

- 这个矩形以第一象限中的一个点 P_i 为左上角。
- 首先，这个矩形不能覆盖第一象限中第一个低于这个点的点 Q_i 。
- 其次，这个矩形右下角的点必须是第四象限的单调栈中的点 S_j 。

- 这个矩形以第一象限中的一个点 P_i 为左上角。
- 首先，这个矩形不能覆盖第一象限中第一个低于这个点的点 Q_i 。
- 其次，这个矩形右下角的点必须是第四象限的单调栈中的点 S_j 。
- 我们可以用一个单调队列维护第四象限中的点 S_j ，遇到一个新的点压入队列，每次用 Q_i 弹队。

总结一下:

总结一下：

- 队列在 OI 中很重要的一点应用就是“单调队列”。

总结一下：

- 队列在 OI 中很重要的一点应用就是“单调队列”。
- 单调队列在滑动窗口 (琪露诺)、决策单调性 Dp(挖油)、背包问题 (Bank notes)、斜率优化等重要问题中有着广泛应用。

总结一下：

- 队列在 OI 中很重要的一点应用就是“单调队列”。
- 单调队列在滑动窗口 (琪露诺)、决策单调性 Dp(挖油)、背包问题 (Bank notes)、斜率优化等重要问题中有着广泛应用。
- 当我们看到一个“先进先出”的集合的时候，就应该想到要使用单调队列。

总结一下：

- 队列在 OI 中很重要的一点应用就是“单调队列”。
- 单调队列在滑动窗口 (琪露诺)、决策单调性 Dp(挖油)、背包问题 (Bank notes)、斜率优化等重要问题中有着广泛应用。
- 当我们看到一个“先进先出”的集合的时候，就应该想到要使用单调队列。
- 同时，我们还学会了斜率优化的相关理论。

1 简介

- 我是谁
- 数据结构是什么

2 正文

- 并查集
 - Anton and Tree (CF734E)
 - 染色问题
 - 方格染色 (luoguP3631)
 - 总结一下
- 队列
 - 琪露诺 (luoguP1725)
 - Bank notes (BZOJ1531)
 - 玩具装箱 (luoguP3195)
 - 挖油 (BZOJ2448)
 - 稻草人 (BZOJ4237)
 - 总结一下
- 栈
 - 双栈排序 (luoguP1155)
 - Harbingers (BZOJ1767)
 - 去月球 (UOJ327)

- 楼房重建 (luoguP4198)
- 一个奇怪的题目
- String Set Queries (CF710F)
- 玄学 (UOJ46)
- String Set Queries 加强版
- 总结一下

● 线段树

- DZY Loves Fibonacci Numbers (CF446C)
- Tree Generator TM (CF1149C)
- HH 的项链 (luoguP1972)
- 神牛的养成计划 (BZOJ4212)
- 火星商店问题 (luoguP4585)
- 树据结构 (51nod1462)
- 总结一下

● 其他数据结构

- 罗马游戏 (BZOJ1455)
- Lomsat gelral (CF600E)
- 总结一下

3 题目推荐

双栈排序 (luoguP1155)

给定一个长度为 n 的排列 $\{a_i\}$ ，问能否借助两个栈的 push 和 pop，使得弹栈序列是 $1, 2, 3 \dots n$ 。

- 我们先回忆一下栈是什么。

- 我们先回忆一下栈是什么。
- 如果存在两个元素 2,1, 只借助一个栈我们有办法交换他们的顺序使他们有序。

- 我们先回忆一下栈是什么。
- 如果存在两个元素 2,1, 只借助一个栈我们有办法交换他们的顺序使他们有序。
- 但是如果三个元素 2,3,1, 只借助一个栈我们就没法交换他们的顺序使他们有序了。

- 我们先回忆一下栈是什么。
- 如果存在两个元素 2,1, 只借助一个栈我们有办法交换他们的顺序使他们有序。
- 但是如果三个元素 2,3,1, 只借助一个栈我们就没法交换他们的顺序使他们有序了。
- 换句话说, 无论有多少个栈, 对于某个有序三元组 2,3,1, 只要 2,3 被压入了同一个栈, 那么弹栈序列永远都不可能有序了。

- 我们先回忆一下栈是什么。
- 如果存在两个元素 2,1, 只借助一个栈我们有办法交换他们的顺序使他们有序。
- 但是如果三个元素 2,3,1, 只借助一个栈我们就没法交换他们的顺序使他们有序了。
- 换句话说, 无论有多少个栈, 对于某个有序三元组 2,3,1, 只要 2,3 被压入了同一个栈, 那么弹栈序列永远都不可能有序了。
- 因此我们的判据可以改为: 对于三元组 $i < j < k$, 如果大小关系为 2,3,1, 则 a_i, a_j 不能进同一个栈。

- 我们先回忆一下栈是什么。
- 如果存在两个元素 2,1, 只借助一个栈我们有办法交换他们的顺序使他们有序。
- 但是如果三个元素 2,3,1, 只借助一个栈我们就没法交换他们的顺序使他们有序了。
- 换句话说, 无论有多少个栈, 对于某个有序三元组 2,3,1, 只要 2,3 被压入了同一个栈, 那么弹栈序列永远都不可能有序了。
- 因此我们的判据可以改为: 对于三元组 $i < j < k$, 如果大小关系为 2,3,1, 则 a_i, a_j 不能进同一个栈。
- 将所有的“不能入同一个栈”的元素对之间连边, 如果构造出的图是二分图, 则还有救。反之没救了。

- 我们先回忆一下栈是什么。
- 如果存在两个元素 2,1, 只借助一个栈我们有办法交换他们的顺序使他们有序。
- 但是如果三个元素 2,3,1, 只借助一个栈我们就没法交换他们的顺序使他们有序了。
- 换句话说, 无论有多少个栈, 对于某个有序三元组 2,3,1, 只要 2,3 被压入了同一个栈, 那么弹栈序列永远都不可能有序了。
- 因此我们的判据可以改为: 对于三元组 $i < j < k$, 如果大小关系为 2,3,1, 则 a_i, a_j 不能进同一个栈。
- 将所有的“不能入同一个栈”的元素对之间连边, 如果构造出的图是二分图, 则还有救。反之没救了。
- 时间复杂度 $O(n^2)$ 。

- 我们先回忆一下栈是什么。
- 如果存在两个元素 2,1, 只借助一个栈我们有办法交换他们的顺序使他们有序。
- 但是如果三个元素 2,3,1, 只借助一个栈我们就没法交换他们的顺序使他们有序了。
- 换句话说, 无论有多少个栈, 对于某个有序三元组 2,3,1, 只要 2,3 被压入了同一个栈, 那么弹栈序列永远都不可能有序了。
- 因此我们的判据可以改为: 对于三元组 $i < j < k$, 如果大小关系为 2,3,1, 则 a_i, a_j 不能进同一个栈。
- 将所有的 “不能入同一个栈” 的元素对之间连边, 如果构造出的图是二分图, 则还有救。反之没救了。
- 时间复杂度 $O(n^2)$ 。
- 还可以优化为 $O(n \log n)$ 甚至 $O(n)$, 难度较大, 详情请见 “题目推荐” 栏目 “port facility” 一题。

Harbingers (BZOJ1767)

给定一棵树，除 1 号以外的每个节点上都有一个快递小哥。每个节点都可能快递要送到 1 号节点去，而快递也总是沿最短路运输。快递每经过一个节点，可以选择：继续由当前的快递小哥运输，或更换这个节点的快递小哥运输。每个快递小哥有 2 个参数，分别为 S_i, T_i 表示接到快递出发前的准备时间，和通过单位距离的时间。现在给出 n ($n \leq 100000$) 个节点和 $n-1$ 条给定正数长度的路，求每个节点的快递运送到 1 号节点的最短时间。

- 假设这棵树是一个以 1 号节点为端点的链，则我们可以很方便地斜率优化：

- 假设这棵树是一个以 1 号节点为端点的链，则我们可以很方便地斜率优化：
- 设 $f[i]$ 为快递从 i 号节点出发到 1 号节点的最短用时。

- 假设这棵树是一个以 1 号节点为端点的链，则我们可以很方便地斜率优化：
- 设 $f[i]$ 为快递从 i 号节点出发到 1 号节点的最短用时。
- 则：

$$f[i] = \min\{f[j] + \text{dis}(i, j)T[i] + S[i] \mid j < i\}$$

- 假设这棵树是一个以 1 号节点为端点的链，则我们可以很方便地斜率优化：
- 设 $f[i]$ 为快递从 i 号节点出发到 1 号节点的最短用时。
- 则：

$$f[i] = \min\{f[j] + \text{dis}(i, j)T[i] + S[i] \mid j < i\}$$

- 通过前缀和优化： $\text{dis}(i, j) = (L[i] - L[j])$

- 假设这棵树是一个以 1 号节点为端点的链，则我们可以很方便地斜率优化：
- 设 $f[i]$ 为快递从 i 号节点出发到 1 号节点的最短用时。
- 则：

$$f[i] = \min\{f[j] + \text{dis}(i, j) T[i] + S[i] \mid j < i\}$$

- 通过前缀和优化： $\text{dis}(i, j) = (L[i] - L[j])$
- 故：

$$f[i] = \min\{f[j] + (L[i] - L[j]) T[i] + S[i] \mid j < i\}$$

- 假设这棵树是一个以 1 号节点为端点的链，则我们可以很方便地斜率优化：
- 设 $f[i]$ 为快递从 i 号节点出发到 1 号节点的最短用时。
- 则：

$$f[i] = \min\{f[j] + \text{dis}(i, j) T[i] + S[i] | j < i\}$$

- 通过前缀和优化： $\text{dis}(i, j) = (L[i] - L[j])$
- 故：

$$f[i] = \min\{f[j] + (L[i] - L[j]) T[i] + S[i] | j < i\}$$

- 写成向量点积的形式（升维打击）：

$$f[i] = \min\{(-L[j], F[j]) \cdot (T[i], 1) | j < i\} + L[i] T[i] + S[i]$$

- 假设这棵树是一个以 1 号节点为端点的链，则我们可以很方便地斜率优化：

- 设 $f[i]$ 为快递从 i 号节点出发到 1 号节点的最短用时。
- 则：

$$f[i] = \min\{f[j] + \text{dis}(i, j) T[i] + S[i] \mid j < i\}$$

- 通过前缀和优化： $\text{dis}(i, j) = (L[i] - L[j])$
- 故：

$$f[i] = \min\{f[j] + (L[i] - L[j]) T[i] + S[i] \mid j < i\}$$

- 写成向量点积的形式（升维打击）：

$$f[i] = \min\{(-L[j], F[j]) \cdot (T[i], 1) \mid j < i\} + L[i] T[i] + S[i]$$

- 直接在 $(-L[j], F[j])$ 构成的凸包上二分即可。

- 假设这棵树是一个以 1 号节点为端点的链，则我们可以很方便地斜率优化：

- 设 $f[i]$ 为快递从 i 号节点出发到 1 号节点的最短用时。
- 则：

$$f[i] = \min\{f[j] + \text{dis}(i, j) T[i] + S[i] \mid j < i\}$$

- 通过前缀和优化： $\text{dis}(i, j) = (L[i] - L[j])$
- 故：

$$f[i] = \min\{f[j] + (L[i] - L[j]) T[i] + S[i] \mid j < i\}$$

- 写成向量点积的形式（升维打击）：

$$f[i] = \min\{(-L[j], F[j]) \cdot (T[i], 1) \mid j < i\} + L[i] T[i] + S[i]$$

- 直接在 $(-L[j], F[j])$ 构成的凸包上二分即可。
- 时间复杂度 $O(n \log n)$

- 由于 $L[j]$ 是单调递增的，我们可以用一个单调栈去维护点集 $(-L[j], F[j])$

- 由于 $L[j]$ 是单调递增的，我们可以用一个单调栈去维护点集 $(-L[j], F[j])$
- 可是如何扩展到树上呢？

- 由于 $L[j]$ 是单调递增的，我们可以用一个单调栈去维护点集 $(-L[j], F[j])$
- 可是如何扩展到树上呢？
- 可以在 dfs 回溯时，恢复所有被弹出栈的元素，但是这样是 $O(n^2)$ 的。怎么卡成 $O(n^2)$ ？

- 由于 $L[j]$ 是单调递增的，我们可以用一个单调栈去维护点集 $(-L[j], F[j])$
- 可是如何扩展到树上呢？
- 可以在 dfs 回溯时，恢复所有被弹出栈的元素，但是这样是 $O(n^2)$ 的。怎么卡成 $O(n^2)$ ？
- 由于每次只会有一个元素被覆盖，以至于从内存中消失。我们只需要恢复这个元素以及栈大小。

- 由于 $L[j]$ 是单调递增的，我们可以用一个单调栈去维护点集 $(-L[j], F[j])$
- 可是如何扩展到树上呢？
- 可以在 dfs 回溯时，恢复所有被弹出栈的元素，但是这样是 $O(n^2)$ 的。怎么卡成 $O(n^2)$ ？
- 由于每次只会有一个元素被覆盖，以至于从内存中消失。我们只需要恢复这个元素以及栈大小。
- 时间复杂度 $O(n \log n)$

去月球 (UOJ327)

定义一个序列的权值为：不断删除相邻且相等的两个元素最多能删掉多少元素。(删掉元素后，后面的元素会填补空位)
给定一个长度为 n 的序列， m 次询问某个连续子序列权值。
($n \leq 10^5, m \leq 2 \times 10^6$)

贪心

能删就删一定最优

贪心

能删就删一定最优

贪心的证明.

两个相等的数能删的时候不删，当且仅当与他们相邻的数字中有与他们相等的数字。
那么删哪两个都是等价的。



贪心

能删就删一定最优

贪心的证明.

两个相等的数能删的时候不删，当且仅当与他们相邻的数字中有与他们相等的数字。
那么删哪两个都是等价的。 □

利用贪心

利用上面的结论，我们可以构造一个栈，依次压入询问的序列中的元素。
每当栈顶有相同元素时，同时弹出栈顶的两个元素。
最后剩下的元素一定是尽量少的。

- 能否利用贪心的策略更快解决问题呢？

- 能否利用贪心的策略更快解决问题呢？
- 受上一题启发，我们知道栈的压与弹的过程可以被离线下来形成一个树形结构。

- 能否利用贪心的策略更快解决问题呢？
- 受上一题启发，我们知道栈的压与弹的过程可以被离线下来形成一个树形结构。
- 我们尝试把整个序列的这个树形结构构建出来。

- 能否利用贪心的策略更快解决问题呢？
- 受上一题启发，我们知道栈的压与弹的过程可以被离线下来形成一个树形结构。
- 我们尝试把整个序列的这个树形结构构建出来。

“距离即元素数” 定理

两点间的距离实际上就等于将序列中他们之间的连续子序列压入栈后栈中最后剩下的元素个数。

- 能否利用贪心的策略更快解决问题呢？
- 受上一题启发，我们知道栈的压与弹的过程可以被离线下来形成一个树形结构。
- 我们尝试把整个序列的这个树形结构构建出来。

“距离即元素数” 定理

两点间的距离实际上就等于将序列中他们之间的连续子序列压入栈后栈中最后剩下的元素个数。

“距离即元素数” 的证明.

两点间的路径的一定先向根走再向叶子走。向根走的路径本来对应着栈顶出现重复元素弹栈的过程。而此刻重复元素根本没有被压入栈，因此没有弹出任何元素，剩余元素数加一。向叶子走的路径对应着不断压入新元素的过程。初始状态不会影响这一段路径的合理性。 □

- 根据“距离即元素数”定理，我们可以构建出原始序列的树形结构，每次用 RMQ 求两点间的距离。

- 根据“距离即元素数”定理，我们可以构建出原始序列的树形结构，每次用 RMQ 求两点间的距离。
- 时间复杂度 $O(n \log n + q)$

楼房重建 (luoguP4198)

定义一个序列的权值为有多少个不同的值是至少一个前缀的最大值。(即该序列对应的单调栈大小)

需要维护一个长度为 n ($n \leq 10^5$) 的序列 A , 支持 m 次修改单个元素, 以及询问某个连续子序列的权值。

- 这是一个经典的题目。

- 这是一个经典的题目。
- 我们定义 $f(l, r, x)$ 为：如果单调栈内已经有唯一的一个元素 x ，再将序列 $A[l \cdots r]$ 压入栈中，最后剩下的元素个数。

- 这是一个经典的题目。
- 我们定义 $f(l, r, x)$ 为：如果单调栈内已经有唯一的一个元素 x ，再将序列 $A[l \cdots r]$ 压入栈中，最后剩下的元素个数。
- 很显然：

$$f(l, r, x) = f(l, mid, x) + f(mid + 1, r, \max(x, \max[l, mid]))$$

- 这是一个经典的题目。
- 我们定义 $f(l, r, x)$ 为：如果单调栈内已经有唯一的一个元素 x ，再将序列 $A[l \cdots r]$ 压入栈中，最后剩下的元素个数。

- 很显然：

$$f(l, r, x) = f(l, mid, x) + f(mid + 1, r, \max(x, \max[l, mid]))$$

- 可以分类讨论一下：

- ▶ $x \leq \max[l, mid]$:

$$f(l, r, x) = f(l, mid, x) + f(mid + 1, r, \max[l, mid])$$

- ▶ $x > \max[l, mid]$: $f(l, r, x) = f(mid + 1, r, x)$

- 这是一个经典的题目。
- 我们定义 $f(l, r, x)$ 为：如果单调栈内已经有唯一的一个元素 x ，再将序列 $A[l \cdots r]$ 压入栈中，最后剩下的元素个数。

- 很显然：

$$f(l, r, x) = f(l, mid, x) + f(mid + 1, r, \max(x, \max[l, mid]))$$

- 可以分类讨论一下：

- ▶ $x \leq \max[l, mid]$:

$$f(l, r, x) = f(l, mid, x) + f(mid + 1, r, \max[l, mid])$$

- ▶ $x > \max[l, mid]$: $f(l, r, x) = f(mid + 1, r, x)$

- 如果我们用线段树维护 f 的值，且在每个节点维护 $f(mid + 1, r, \max[l, mid])$ 的值，我们就可以做到 $O(\log^2 n)$ 修改和查询了。

- 这是一个经典的题目。
- 我们定义 $f(l, r, x)$ 为：如果单调栈内已经有唯一的一个元素 x ，再将序列 $A[l \cdots r]$ 压入栈中，最后剩下的元素个数。

- 很显然：

$$f(l, r, x) = f(l, mid, x) + f(mid + 1, r, \max(x, \max[l, mid]))$$

- 可以分类讨论一下：

- ▶ $x \leq \max[l, mid]$:

$$f(l, r, x) = f(l, mid, x) + f(mid + 1, r, \max[l, mid])$$

- ▶ $x > \max[l, mid]$: $f(l, r, x) = f(mid + 1, r, x)$

- 如果我们用线段树维护 f 的值，且在每个节点维护 $f(mid + 1, r, \max[l, mid])$ 的值，我们就可以做到 $O(\log^2 n)$ 修改和查询了。
- 画个图看一下更好。

一个奇怪的题目

要求维护一个由 n ($n \leq 10^5$) 个单调栈组成的序列。
支持 m 次区间 $push(x)$ 和单点查询单调栈的大小。

- 对这个序列进行升维打击：第 i 次区间 $push(x)$ 操作，在该区间高度为 i 处写下数字 x 。

- 对这个序列进行升维打击：第 i 次区间 $push(x)$ 操作，在该区间高度为 i 处写下数字 x 。
- 每次询问等于是询问某点处的楼房重建问题。

- 对这个序列进行升维打击：第 i 次区间 $push(x)$ 操作，在该区间高度为 i 处写下数字 x 。
- 每次询问等于是询问某点处的楼房重建问题。
- 因此将询问和修改离线下来，从左往右扫描，用楼房重建问题中的线段树维护答案。

- 对这个序列进行升维打击：第 i 次区间 $push(x)$ 操作，在该区间高度为 i 处写下数字 x 。
- 每次询问等于是询问某点处的楼房重建问题。
- 因此将询问和修改离线下来，从左往右扫描，用楼房重建问题中的线段树维护答案。
- 时间复杂度 $O(n \log m)$

String Set Queries (CF710F)

维护一个字符串集合，支持三种操作：

- 加字符串。
- 删字符串。
- 查询集合中的所有字符串在给定的模板串中出现的次数。

操作数 $m \leq 3 \times 10^5$ ，输入字符串总长度 $\sum |s_i| \leq 3 \times 10^5$ 。强制在线。

String Set Queries (CF710F)

维护一个字符串集合，支持三种操作：

- 加字符串。
- 删字符串。
- 查询集合中的所有字符串在给定的模板串中出现的次数。

操作数 $m \leq 3 \times 10^5$ ，输入字符串总长度 $\sum |s_i| \leq 3 \times 10^5$ 。强制在线。

- 如果允许离线查询给定的字符串集合中所有串在模板串中的出现次数之和，只需要使用 AC 自动机就可以了。

String Set Queries (CF710F)

维护一个字符串集合，支持三种操作：

- 加字符串。
- 删字符串。
- 查询集合中的所有字符串在给定的模板串中出现的次数。

操作数 $m \leq 3 \times 10^5$ ，输入字符串总长度 $\sum |s_i| \leq 3 \times 10^5$ 。强制在线。

- 如果允许离线查询给定的字符串集合中所有串在模板串中的出现次数之和，只需要使用 AC 自动机就可以了。
- 如果需要支持在线，我们需要实现一个在线的 AC 自动机。但是这种东西是不存在的。

String Set Queries (CF710F)

维护一个字符串集合，支持三种操作：

- 加字符串。
- 删字符串。
- 查询集合中的所有字符串在给定的模板串中出现的次数。

操作数 $m \leq 3 \times 10^5$ ，输入字符串总长度 $\sum |s_i| \leq 3 \times 10^5$ 。强制在线。

- 如果允许离线查询给定的字符串集合中所有串在模板串中的出现次数之和，只需要使用 AC 自动机就可以了。
- 如果需要在支持在线，我们需要实现一个在线的 AC 自动机。但是这种东西是不存在的。
- 能否用多个 AC 自动机的线性组合来组成一个在线 AC 自动机呢？

Outline

简介

我是谁

数据结构是什么

正文

并查集

队列

栈

线段树

其他数据结构

题目推荐

- 答案是肯定的。

- 答案是肯定的。
- 我们定义一个单调栈，栈内维护若干个 AC 自动机。在任意时刻需要满足靠近栈顶的 AC 自动机大小（包含的串的数目）小于远离栈顶的 AC 自动机。

- 答案是肯定的。
- 我们定义一个单调栈，栈内维护若干个 AC 自动机。在任意时刻需要满足靠近栈顶的 AC 自动机大小（包含的串的数目）小于远离栈顶的 AC 自动机。
- 每加入或删除字符串，都将该字符串单独构建一个 AC 自动机压入栈中。

- 答案是肯定的。
- 我们定义一个单调栈，栈内维护若干个 AC 自动机。在任意时刻需要满足靠近栈顶的 AC 自动机大小（包含的串的数目）小于远离栈顶的 AC 自动机。
- 每加入或删除字符串，都将该字符串单独构建一个 AC 自动机压入栈中。
- 当栈的单调性被破坏时，需要自顶向下合并 AC 自动机并重构。

- 答案是肯定的。
- 我们定义一个单调栈，栈内维护若干个 AC 自动机。在任意时刻需要满足靠近栈顶的 AC 自动机大小（包含的串的数目）小于远离栈顶的 AC 自动机。
- 每加入或删除字符串，都将该字符串单独构建一个 AC 自动机压入栈中。
- 当栈的单调性被破坏时，需要自顶向下合并 AC 自动机并重构。
- 不难证明，栈内的元素大小都是二的整数幂，且一个字符串至多经历 $O(\log m)$ 次 AC 自动机的合并。
- 所以复杂度为 $O(\log m \sum |s_i|)$

玄学 (UOJ46)

给定模数 m , 一个数列 $\{a_i\}$, 一个函数序列 $\{f_i\}$ 。第 i 个函数为: 将数列 $[l_i, r_i]$ 区间内的元素 x 变为 $(a_i x + b_i) \bmod m$ 。你需要实现:

- 在函数序列末尾插入一个新的函数。
- 询问: 如果对数列依次应用函数 $f_l, f_{l+1}, \dots, f_r$ 之后, 第 k 位的值变为了多少。(询问不对数列产生影响)

数列长度 $n \leq 10^5$, 询问次数 $q \leq 6 \times 10^5$ 。

- 注意到题目中的函数是不可逆且不满足交换律的，因此不能使用前缀和相减的方法去求答案。

- 注意到题目中的函数是不可逆且不满足交换律的，因此不能使用前缀和相减的方法去求答案。
- 但是该函数满足结合律。

- 注意到题目中的函数是不可逆且不满足交换律的，因此不能使用前缀和相减的方法去求答案。
- 但是该函数满足结合律。
- 我们可以套用上一题二进制分组的方法去构建一棵类似于时间线段树的结构。线段树的父节点由子节点归并得到。

- 注意到题目中的函数是不可逆且不满足交换律的，因此不能使用前缀和相减的方法去求答案。
- 但是该函数满足结合律。
- 我们可以套用上一题二进制分组的方法去构建一棵类似于时间线段树的结构。线段树的父节点由子节点归并得到。
- 查询一段区间中所有函数对原数列的作用，根据结合律，就只需要查询线段树上 $\log n$ 个区间对数列的作用。

- 注意到题目中的函数是不可逆且不满足交换律的，因此不能使用前缀和相减的方法去求答案。
- 但是该函数满足结合律。
- 我们可以套用上一题二进制分组的方法去构建一棵类似于时间线段树的结构。线段树的父节点由子节点归并得到。
- 查询一段区间中所有函数对原数列的作用，根据结合律，就只需要查询线段树上 $\log n$ 个区间对数列的作用。
- 而一个长度为 len 的线段树区间对原数列的作用可以表示为 $O(len)$ 个本质不同的函数分别作用于原数列的几个不相交区间。

- 注意到题目中的函数是不可逆且不满足交换律的，因此不能使用前缀和相减的方法去求答案。
- 但是该函数满足结合律。
- 我们可以套用上一题二进制分组的方法去构建一棵类似于时间线段树的结构。线段树的父节点由子节点归并得到。
- 查询一段区间中所有函数对原数列的作用，根据结合律，就只需要查询线段树上 $\log n$ 个区间对数列的作用。
- 而一个长度为 len 的线段树区间对原数列的作用可以表示为 $O(len)$ 个本质不同的函数分别作用于原数列的几个不相交区间。
- 因此某个线段树区间对原数列某一位的作用可以在该线段树区间内二分得到。

String Set Queries 加强版

你需要维护一个元素是字符串的序列，初始为空，并支持：

- 在第 i 个元素前面插入一个元素，后面的元素编号增加 1。
- 删除第 i 个元素，后面的元素编号减 1。
- 给定一个模板串，查询 $[l, r]$ 区间中的每个串在模板串中出现次数之和。

询问次数 $q \leq 100000$ 。

- 我们同样维护一个元素为 AC 自动机的栈。

- 我们同样维护一个元素为 AC 自动机的栈。
- 在位置 p 插入和删除字符串 s 时，在栈顶插入一个三元组 $(p, s, \pm 1)$ 。

- 我们同样维护一个元素为 AC 自动机的栈。
- 在位置 p 插入和删除字符串 s 时，在栈顶插入一个三元组 $(p, s, \pm 1)$ 。
- 合并栈顶元素时，将两个 AC 自动机合并，且三元组的第一维进行归并。需要保证：栈的每一层内的三元组的 p 都表示“删除该层以上的元素后，该三元组在序列中的位置排名”。

- 我们同样维护一个元素为 AC 自动机的栈。
- 在位置 p 插入和删除字符串 s 时，在栈顶插入一个三元组 $(p, s, \pm 1)$ 。
- 合并栈顶元素时，将两个 AC 自动机合并，且三元组的第一维进行归并。需要保证：栈的每一层内的三元组的 p 都表示“删除该层以上的元素后，该三元组在序列中的位置排名”。
- 查询时，自顶向下在每个 AC 自动机内查询。由于需要查询 $[l, r]$ 区间内的出现次数，每个 AC 自动机内我们需要用线段树合并维护子树出现次数。

- 我们同样维护一个元素为 AC 自动机的栈。
- 在位置 p 插入和删除字符串 s 时，在栈顶插入一个三元组 $(p, s, \pm 1)$ 。
- 合并栈顶元素时，将两个 AC 自动机合并，且三元组的第一维进行归并。需要保证：栈的每一层内的三元组的 p 都表示“删除该层以上的元素后，该三元组在序列中的位置排名”。
- 查询时，自顶向下在每个 AC 自动机内查询。由于需要查询 $[l, r]$ 区间内的出现次数，每个 AC 自动机内我们需要用线段树合并维护子树出现次数。
- 在栈的某一层查询完成后，我们需要将 l, r 进行相应的增减，以排除掉已经被查询过的串所占据的位置。

总结一下:

总结一下：

- 栈在 OI 中也有着广泛且深入的应用。

总结一下：

- 栈在 OI 中也有着广泛且深入的应用。
- 单调栈可以用于优化 Dp (Harbingers)。

总结一下:

- 栈在 OI 中也有着广泛且深入的应用。
- 单调栈可以用于优化 Dp (Harbingers)。
- 树的 dfs 也可以看成是在栈内不断压入元素和弹出元素的过程 (去月球)。

总结一下:

- 栈在 OI 中也有着广泛且深入的应用。
- 单调栈可以用于优化 Dp (Harbingers)。
- 树的 dfs 也可以看成是在栈内不断压入元素和弹出元素的过程 (去月球)。
- 楼房重建类问题要求你用线段树去维护一个动态的栈 (楼房重建)。

总结一下:

- 栈在 OI 中也有着广泛且深入的应用。
- 单调栈可以用于优化 Dp (Harbingers)。
- 树的 dfs 也可以看成是在栈内不断压入元素和弹出元素的过程 (去月球)。
- 楼房重建类问题要求你用线段树去维护一个动态的栈 (楼房重建)。
- 利用单调栈套数据结构可以实现离线可加数据结构的在线化 (String Set Queries)。

1 简介

- 我是谁
- 数据结构是什么

2 正文

- 并查集
 - Anton and Tree (CF734E)
 - 染色问题
 - 方格染色 (luoguP3631)
 - 总结一下
- 队列
 - 琪露诺 (luoguP1725)
 - Bank notes (BZOJ1531)
 - 玩具装箱 (luoguP3195)
 - 挖油 (BZOJ2448)
 - 稻草人 (BZOJ4237)
 - 总结一下
- 栈
 - 双栈排序 (luoguP1155)
 - Harbingers (BZOJ1767)
 - 去月球 (UOJ327)

- 楼房重建 (luoguP4198)
- 一个奇怪的题目
- String Set Queries (CF710F)
- 玄学 (UOJ46)
- String Set Queries 加强版
- 总结一下

● 线段树

- DZY Loves Fibonacci Numbers (CF446C)
- Tree Generator TM (CF1149C)
- HH 的项链 (luoguP1972)
- 神牛的养成计划 (BZOJ4212)
- 火星商店问题 (luoguP4585)
- 树据结构 (51nod1462)
- 总结一下

● 其他数据结构

- 罗马游戏 (BZOJ1455)
- Lomsat gelral (CF600E)
- 总结一下

3 题目推荐

DZY Loves Fibonacci Numbers (CF446C)

我们定义斐波那契数列为

$fib_1 = fib_2 = 1, fib_i = fib_{i-1} + fib_{i-2} (i \geq 3)$, 你需要维护一个序列 a_i , 并支持:

- 对于 $i \in [l, r]$ 都进行 $a_i += fib_{i-l+1}$
- 求 $(\sum_{i=l}^r a_i) \bmod 10^9$

其中序列长度 $n \leq 3 \times 10^5$, 操作次数 $m \leq 3 \times 10^5$ 。

Outline

简介

我是谁

数据结构是什么

正文

并查集

队列

栈

线段树

其他数据结构

题目推荐

- 如果问题改为：区间加等差数列，区间求和，怎么做呢？

Outline

简介

我是谁

数据结构是什么

正文

并查集

队列

栈

线段树

其他数据结构

题目推荐

- 如果问题改为：区间加等差数列，区间求和，怎么做呢？
- 我们知道斐波那契数列是可以通过矩阵乘法递推的。

Outline

简介

我是谁

数据结构是什么

正文

并查集

队列

栈

线段树

其他数据结构

题目推荐

- 如果问题改为：区间加等差数列，区间求和，怎么做呢？
- 我们知道斐波那契数列是可以通过矩阵乘法递推的。
- 而每一次区间加操作相当于给原序列加上了一个矩阵的等比数列。

- 如果问题改为：区间加等差数列，区间求和，怎么做呢？
- 我们知道斐波那契数列是可以通过矩阵乘法递推的。
- 而每一次区间加操作相当于给原序列加上了一个矩阵的等比数列。
- 矩阵的等比数列求和也是很容易的，比如倍增法求和。由于斐波那契数列的矩阵可逆，我们也可以用“等比数列求和公式”。

- 如果问题改为：区间加等差数列，区间求和，怎么做呢？
- 我们知道斐波那契数列是可以通过矩阵乘法递推的。
- 而每一次区间加操作相当于给原序列加上了一个矩阵的等比数列。
- 矩阵的等比数列求和也是很容易的，比如倍增法求和。由于斐波那契数列的矩阵可逆，我们也可以用“等比数列求和公式”。
- 因此，我们可以用线段树去完成“矩阵的区间加等比数列”和“矩阵的区间求和”的问题。

- 如果问题改为：区间加等差数列，区间求和，怎么做呢？
- 我们知道斐波那契数列是可以通过矩阵乘法递推的。
- 而每一次区间加操作相当于给原序列加上了一个矩阵的等比数列。
- 矩阵的等比数列求和也是很容易的，比如倍增法求和。由于斐波那契数列的矩阵可逆，我们也可以用“等比数列求和公式”。
- 因此，我们可以用线段树去完成“矩阵的区间加等比数列”和“矩阵的区间求和”的问题。
- 时间复杂度 $O(n \log^2 n)$ 。

树据结构 (51nod1462)

给出一棵 n 个点的树，每个点有两个权值 v, t
有 Q 个操作，有两种操作：

- 将 x 到根上的路径上的点的 v 值都加上 d 。
- 将 x 到根上的路径上的点的 t 值都加上每个点的 $v \times d$ 。

最后求出所有点的 t 值

- 根据套路，我们会把这棵树进行树链剖分，然后用线段树维护每个点的 t, v 。

- 根据套路，我们会把这棵树进行树链剖分，然后用线段树维护每个点的 t, v 。
- 最大的问题其实就是如何维护每个节点两个值。

- 根据套路，我们会把这棵树进行树链剖分，然后用线段树维护每个点的 t, v 。
- 最大的问题其实就是要如何维护每个节点两个值。
- 乍一看，两个值的更新先后顺序非常重要，很难理清思路。

- 根据套路，我们会把这棵树进行树链剖分，然后用线段树维护每个点的 t, v 。
- 最大的问题其实就是要如何维护每个节点两个值。
- 乍一看，两个值的更新先后顺序非常重要，很难理清思路。
- 但是再一想，如果把这两个值组合成一个向量，则两个修改操作都可以认为是给向量左乘了一个矩阵。

- 根据套路，我们会把这棵树进行树链剖分，然后用线段树维护每个点的 t, v 。
- 最大的问题其实就是要如何维护每个节点两个值。
- 乍一看，两个值的更新先后顺序非常重要，很难理清思路。
- 但是再一想，如果把这两个值组合成一个向量，则两个修改操作都可以认为是给向量左乘了一个矩阵。
- 第一个操作：

$$\begin{bmatrix} v' \\ t' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & d \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ t \\ 1 \end{bmatrix}$$

- 根据套路，我们会把这棵树进行树链剖分，然后用线段树维护每个点的 t, v 。
- 最大的问题其实就是要如何维护每个节点两个值。
- 乍一看，两个值的更新先后顺序非常重要，很难理清思路。
- 但是再一想，如果把这两个值组合成一个向量，则两个修改操作都可以认为是给向量左乘了一个矩阵。
- 第一个操作：

$$\begin{bmatrix} v' \\ t' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & d \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ t \\ 1 \end{bmatrix}$$

- 第二个操作：

$$\begin{bmatrix} v' \\ t' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ d & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ t \\ 1 \end{bmatrix}$$

- 根据套路，我们会把这棵树进行树链剖分，然后用线段树维护每个点的 t, v 。
- 最大的问题其实就是要如何维护每个节点两个值。
- 乍一看，两个值的更新先后顺序非常重要，很难理清思路。
- 但是再一想，如果把这两个值组合成一个向量，则两个修改操作都可以认为是给向量左乘了一个矩阵。
- 第一个操作：

$$\begin{bmatrix} v' \\ t' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & d \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ t \\ 1 \end{bmatrix}$$

- 第二个操作：

$$\begin{bmatrix} v' \\ t' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ d & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ t \\ 1 \end{bmatrix}$$

- 这样，我们就只需要一个维护区间乘法的线段树就行了。

Tree Generator TM (CF1149C)

给出一棵 $n(n \leq 2 \times 10^5)$ 个节点的树的括号序，有 $m(m \leq 2 \times 10^5)$ 个操作，每次交换两个括号，保证交换后括号序仍然合法，输出每次操作后（包括未操作时）树的直径。

Outline

简介

我是谁

数据结构是什么

正文

并查集

队列

栈

线段树

其他数据结构

题目推荐

- 树的直径即括号序列中一个删去匹配括号后剩余括号最多的子串。

- 树的直径即括号序列中一个删去匹配括号后剩余括号最多的子串。
- 如果我们令左括号为 -1 ，右括号为 $+1$ ， S_i 为序列的前缀和，则答案可以表示为：

$$\begin{aligned} & \max\{(S_k - S_j) - (S_j - S_i) | i \leq j \leq k\} \\ &= \max\{S_k - 2S_j + S_i | i \leq j \leq k\} \end{aligned}$$

$$(((i))) \dots ((j)) \dots (((k)))$$

图：括号序列

Outline

简介

我是谁

数据结构是什么

正文

并查集

队列

栈

线段树

其他数据结构

题目推荐

- 能否用线段树动态维护这个函数的最大值呢？

- 能否用线段树动态维护这个函数的最大值呢?
- 我们考虑 i, j, k 相对于线段树区间的位置。
 - ▶ $i \leq j \leq k \leq mid$: 该区间内的答案为左子树的答案。
 - ▶ $i \leq j \leq mid \leq k$: 该区间内的答案为左子树的 $\max\{-2S_j + S_i\}$ 加右子树的 $\max\{S_i\}$ 。
 - ▶ $i \leq mid \leq j \leq k$: 该区间内的答案为左子树的 $\max\{S_i\}$ 加右子树的 $\max\{-2S_j + S_i\}$ 。
 - ▶ $i \leq j \leq k \leq mid$: 该区间内的答案为右子树的答案。

- 能否用线段树动态维护这个函数的最大值呢?
- 我们考虑 i, j, k 相对于线段树区间的位置。
 - ▶ $i \leq j \leq k \leq mid$: 该区间内的答案为左子树的答案。
 - ▶ $i \leq j \leq mid \leq k$: 该区间内的答案为左子树的 $\max\{-2S_j + S_i\}$ 加右子树的 $\max\{S_i\}$ 。
 - ▶ $i \leq mid \leq j \leq k$: 该区间内的答案为左子树的 $\max\{S_i\}$ 加右子树的 $\max\{-2S_j + S_i\}$ 。
 - ▶ $i \leq j \leq k \leq mid$: 该区间内的答案为右子树的答案。
- 在每个节点维护 $\max\{S_i\}, \max\{-2S_j + S_i\}, \max\{S_k - 2S_j + S_i\}$ 即可。

HH 的项链 (luoguP1972)

给定一个长度为 n 的序列 $\{a_i\}$ ，以及 m 个询问，每次询问区间 $[l, r]$ 中不同元素的个数。

Outline

简介

我是谁

数据结构是什么

正文

并查集

队列

栈

线段树

其他数据结构

题目推荐

- 这道题可以使用莫队算法解决。

Outline

简介

我是谁

数据结构是什么

正文

并查集

队列

栈

线段树

其他数据结构

题目推荐

- 这道题可以使用莫队算法解决。
- 但是换个思路会更简单一些。

Outline

简介

我是谁

数据结构是什么

正文

并查集

队列

栈

线段树

其他数据结构

题目推荐

- 这道题可以使用莫队算法解决。
- 但是换个思路会更简单一些。
- 我们对问题进行“升维打击”：

- 这道题可以使用莫队算法解决。
- 但是换个思路会更简单一些。
- 我们对问题进行“升维打击”：
- 对于序列的每个元素 a_i ，如果下一个和他相同的元素在 a_j 处出现，则在一个二维平面内插入点 (i, j) 。

- 这道题可以使用莫队算法解决。
- 但是换个思路会更简单一些。
- 我们对问题进行“升维打击”：
- 对于序列的每个元素 a_i ，如果下一个和他相同的元素在 a_j 处出现，则在一个二维平面内插入点 (i, j) 。
- 每一个询问可以看成是：查询矩形 $x \in [1, l], y \in [l, r]$ 内有多少个点。

- 这道题可以使用莫队算法解决。
- 但是换个思路会更简单一些。
- 我们对问题进行“升维打击”：
- 对于序列的每个元素 a_i ，如果下一个和他相同的元素在 a_j 处出现，则在一个二维平面内插入点 (i, j) 。
- 每一个询问可以看成是：查询矩形 $x \in [1, l], y \in [l, r]$ 内有多少个点。
- 可以使用经典的线段树 + 扫描线解决。

- 这道题可以使用莫队算法解决。
- 但是换个思路会更简单一些。
- 我们对问题进行“升维打击”：
- 对于序列的每个元素 a_i ，如果下一个和他相同的元素在 a_j 处出现，则在一个二维平面内插入点 (i, j) 。
- 每一个询问可以看成是：查询矩形 $x \in [1, l], y \in [l, r]$ 内有多少个点。
- 可以使用经典的线段树 + 扫描线解决。
- 时间复杂度 $O((n + m) \log n)$ 。

神牛的养成计划 (BZOJ4212)

给你 $n(n \leq 2000)$ 个字符串 S_i , 以及 $q(q \leq 10^5)$ 个询问串二元组 (A_i, B_i) , 每次询问有多少个给定的字符串 S 满足 A_i 是前缀, B_i 是后缀。

前缀与 fail 树

如果字符串 A 是 B 的前缀，那么 AC 自动机中， A 一定是 B 在 fail 树上的祖先。

前缀与 fail 树

如果字符串 A 是 B 的前缀，那么 AC 自动机中， A 一定是 B 在 fail 树上的祖先。

- 根据上面的定理，我们可以使用 AC 自动机来限定前缀关系。

前缀与 fail 树

如果字符串 A 是 B 的前缀，那么 AC 自动机中， A 一定是 B 在 fail 树上的祖先。

- 根据上面的定理，我们可以使用 AC 自动机来限定前缀关系。
- 同理，后缀关系可以用反串的 AC 自动机来限定。

前缀与 fail 树

如果字符串 A 是 B 的前缀，那么 AC 自动机中， A 一定是 B 在 fail 树上的祖先。

- 根据上面的定理，我们可以使用 AC 自动机来限定前缀关系。
- 同理，后缀关系可以用反串的 AC 自动机来限定。
- 现在我们的任务变成了，求有多少个串 S 在正串的 AC 自动机上属于 fail 树的某棵子树，在反串 AC 自动机中也属于某棵子树。

前缀与 fail 树

如果字符串 A 是 B 的前缀，那么 AC 自动机中， A 一定是 B 在 fail 树上的祖先。

- 根据上面的定理，我们可以使用 AC 自动机来限定前缀关系。
- 同理，后缀关系可以用反串的 AC 自动机来限定。
- 现在我们的任务变成了，求有多少个串 S 在正串的 AC 自动机上属于 fail 树的某棵子树，在反串的 AC 自动机中也属于某棵子树。
- 这就是经典的子树求交问题。

子树求交问题

给定两棵树 A, B ，点的编号都是从 1 到 n 。

给定 a, b 两点，现在问有多少点 x 在 A 树中在点 a 的子树内，在 B 树中在点 b 的子树内。

子树求交问题

给定两棵树 A, B ，点的编号都是从 1 到 n 。

给定 a, b 两点，现在问有多少点 x 在 A 树中在点 a 的子树内，在 B 树中在点 b 的子树内。

- 我们可以先把询问 (a_i, b_i) 离线下来，挂在 A 树上。

子树求交问题

给定两棵树 A, B ，点的编号都是从 1 到 n 。

给定 a, b 两点，现在问有多少点 x 在 A 树中在点 a 的子树内，在 B 树中在点 b 的子树内。

- 我们可以先把询问 (a_i, b_i) 离线下来，挂在 A 树上。
- 然后对 A 树进行遍历。每新到一个点 x ，处理挂在这个点处的询问 (x, b_i) ：询问 B 树中 b_i 的子树和。然后，将 B 树编号为 x 的点加一。

子树求交问题

给定两棵树 A, B ，点的编号都是从 1 到 n 。

给定 a, b 两点，现在问有多少点 x 在 A 树中在点 a 的子树内，在 B 树中在点 b 的子树内。

- 我们可以先把询问 (a_i, b_i) 离线下来，挂在 A 树上。
- 然后对 A 树进行遍历。每新到一个点 x ，处理挂在这个点处的询问 (x, b_i) ：询问 B 树中 b_i 的子树和。然后，将 B 树编号为 x 的点加一。
- 每离开一个点，再处理一遍挂在这个点处的询问：询问 B 树中 b_i 的子树和。

子树求交问题

给定两棵树 A, B ，点的编号都是从 1 到 n 。

给定 a, b 两点，现在问有多少点 x 在 A 树中在点 a 的子树内，在 B 树中在点 b 的子树内。

- 我们可以先把询问 (a_i, b_i) 离线下来，挂在 A 树上。
- 然后对 A 树进行遍历。每新到一个点 x ，处理挂在这个点处的询问 (x, b_i) ：询问 B 树中 b_i 的子树和。然后，将 B 树编号为 x 的点加一。
- 每离开一个点，再处理一遍挂在这个点处的询问：询问 B 树中 b_i 的子树和。
- 两次询问的答案之差即为原问题的答案。

子树求交问题

给定两棵树 A, B ，点的编号都是从 1 到 n 。

给定 a, b 两点，现在问有多少点 x 在 A 树中在点 a 的子树内，在 B 树中在点 b 的子树内。

- 我们可以先把询问 (a_i, b_i) 离线下来，挂在 A 树上。
- 然后对 A 树进行遍历。每新到一个点 x ，处理挂在这个点处的询问 (x, b_i) ：询问 B 树中 b_i 的子树和。然后，将 B 树编号为 x 的点加一。
- 每离开一个点，再处理一遍挂在这个点处的询问：询问 B 树中 b_i 的子树和。
- 两次询问的答案之差即为原问题的答案。
- 维护子树和的任务可以交给线段树，当然树状数组也是可以的。

子树求交问题

给定两棵树 A, B ，点的编号都是从 1 到 n 。

给定 a, b 两点，现在问有多少点 x 在 A 树中在点 a 的子树内，在 B 树中在点 b 的子树内。

- 我们可以先把询问 (a_i, b_i) 离线下来，挂在 A 树上。
- 然后对 A 树进行遍历。每新到一个点 x ，处理挂在这个点处的询问 (x, b_i) ：询问 B 树中 b_i 的子树和。然后，将 B 树编号为 x 的点加一。
- 每离开一个点，再处理一遍挂在这个点处的询问：询问 B 树中 b_i 的子树和。
- 两次询问的答案之差即为原问题的答案。
- 维护子树和的任务可以交给线段树，当然树状数组也是可以的。
- 时间复杂度 $O(n \log n)$

火星商店问题 (luoguP4585)

你需要维护一个元素是 $\text{set}<\text{int}>$ 的序列。并支持：

- 在第 p 个 $\text{set}<\text{int}>$ 内插入 x
- 询问编号在 $[l, r]$ 的 $\text{set}<\text{int}>$ 中，插入时间在 t 以后的元素中，与 x 异或的最大值。

序列长度为 n ，询问和修改次数为 m ，允许离线。

- 众所周知，查询异或最大值的数据结构为字典树。

Outline

简介

我是谁

数据结构是什么

正文

并查集

队列

栈

线段树

其他数据结构

题目推荐

- 众所周知，查询异或最大值的数据结构为字典树。
- 现在我们的查询需要满足两个关键字：set 的编号和插入的时间。

- 众所周知，查询异或最大值的数据结构为字典树。
- 现在我们的查询需要满足两个关键字：set 的编号和插入的时间。
- 实际上很容易做到：我们可以对序列维护一个线段树，线段树的每个节点维护一棵可持久化字典树。

- 众所周知，查询异或最大值的数据结构为字典树。
- 现在我们的查询需要满足两个关键字：set 的编号和插入的时间。
- 实际上很容易做到：我们可以对序列维护一个线段树，线段树的每个节点维护一棵可持久化字典树。
- 每次询问首先找到对应的线段树区间，再通过可持久化字典树的差分得到一个时间区间内插入的元素组成的字典树。在这个字典树上查询异或最大值即可。

- 众所周知，查询异或最大值的数据结构为字典树。
- 现在我们的查询需要满足两个关键字：set 的编号和插入的时间。
- 实际上很容易做到：我们可以对序列维护一个线段树，线段树的每个节点维护一棵可持久化字典树。
- 每次询问首先找到对应的线段树区间，再通过可持久化字典树的差分得到一个时间区间内插入的元素组成的字典树。在这个字典树上查询异或最大值即可。
- 答案就是每个线段树区间查询得到的最大值的最大值。

- 众所周知，查询异或最大值的数据结构为字典树。
- 现在我们的查询需要满足两个关键字：set 的编号和插入的时间。
- 实际上很容易做到：我们可以对序列维护一个线段树，线段树的每个节点维护一棵可持久化字典树。
- 每次询问首先找到对应的线段树区间，再通过可持久化字典树的差分得到一个时间区间内插入的元素组成的字典树。在这个字典树上查询异或最大值即可。
- 答案就是每个线段树区间查询得到的最大值的最大值。
- 时间复杂度 $O(m \log V \log n)$ ，其中 V 是序列元素的最大值。

总结一下:

Outline

简介

我是谁

数据结构是什么

正文

并查集

队列

栈

线段树

其他数据结构

题目推荐

总结一下：

- 线段树作为最简单的高级数据结构，在 OI 中有着广泛的应用。

总结一下：

- 线段树作为最简单的高级数据结构，在 OI 中有着广泛的应用。
- 我们可以通过建立有用的域来制造出维护特定值的线段树 (Tree Generator TM) 。

总结一下：

- 线段树作为最简单的高级数据结构，在 OI 中有着广泛的应用。
- 我们可以通过建立有用的域来制造出维护特定值的线段树 (Tree Generator TM) 。
- 我们还可以把线段树当成一个模板，想使用的时候就套用他来解决需要动态修改和询问的问题 (神牛的养成计划)。

总结一下:

- 线段树作为最简单的高级数据结构, 在 OI 中有着广泛的应用。
- 我们可以通过建立有用的域来制造出维护特定值的线段树 (Tree Generator TM) 。
- 我们还可以把线段树当成一个模板, 想使用的时候就套用他来解决需要动态修改和询问的问题 (神牛的养成计划)。
- 线段树也可以与其他数据结构嵌套, 来解决更复杂的问题 (火星商店问题)。

总结一下：

- 线段树作为最简单的高级数据结构，在 OI 中有着广泛的应用。
- 我们可以通过建立有用的域来制造出维护特定值的线段树 (Tree Generator TM) 。
- 我们还可以把线段树当成一个模板，想使用的时候就套用他来解决需要动态修改和询问的问题 (神牛的养成计划)。
- 线段树也可以与其他数据结构嵌套，来解决更复杂的问题 (火星商店问题)。
- 另外，许多线段树可以解决的问题都可以用树状数组解决，如《火星商店问题》中的线段树可以替换为树状数组。

总结一下:

- 线段树作为最简单的高级数据结构, 在 OI 中有着广泛的应用。
- 我们可以通过建立有用的域来制造出维护特定值的线段树 (Tree Generator TM) 。
- 我们还可以把线段树当成一个模板, 想使用的时候就套用他来解决需要动态修改和询问的问题 (神牛的养成计划)。
- 线段树也可以与其他数据结构嵌套, 来解决更复杂的问题 (火星商店问题)。
- 另外, 许多线段树可以解决的问题都可以用树状数组解决, 如《火星商店问题》中的线段树可以替换为树状数组。
- 什么样的问题可以用树状数组解决呢?

总结一下:

- 线段树作为最简单的高级数据结构, 在 OI 中有着广泛的应用。
- 我们可以通过建立有用的域来制造出维护特定值的线段树 (Tree Generator TM) 。
- 我们还可以把线段树当成一个模板, 想使用的时候就套用他来解决需要动态修改和询问的问题 (神牛的养成计划)。
- 线段树也可以与其他数据结构嵌套, 来解决更复杂的问题 (火星商店问题)。
- 另外, 许多线段树可以解决的问题都可以用树状数组解决, 如《火星商店问题》中的线段树可以替换为树状数组。
- 什么样的问题可以用树状数组解决呢?
- 前缀查询类问题, 或者询问结果可逆的区间查询类问题。

1 简介

- 我是谁
- 数据结构是什么

2 正文

- 并查集
 - Anton and Tree (CF734E)
 - 染色问题
 - 方格染色 (luoguP3631)
 - 总结一下
- 队列
 - 琪露诺 (luoguP1725)
 - Bank notes (BZOJ1531)
 - 玩具装箱 (luoguP3195)
 - 挖油 (BZOJ2448)
 - 稻草人 (BZOJ4237)
 - 总结一下
- 栈
 - 双栈排序 (luoguP1155)
 - Harbingers (BZOJ1767)
 - 去月球 (UOJ327)

- 楼房重建 (luoguP4198)
- 一个奇怪的题目
- String Set Queries (CF710F)
- 玄学 (UOJ46)
- String Set Queries 加强版
- 总结一下
- 线段树
 - DZY Loves Fibonacci Numbers (CF446C)
 - Tree Generator TM (CF1149C)
 - HH 的项链 (luoguP1972)
 - 神牛的养成计划 (BZOJ4212)
 - 火星商店问题 (luoguP4585)
 - 树据结构 (51nod1462)
 - 总结一下
- 其他数据结构
 - 罗马游戏 (BZOJ1455)
 - Lomsat gelral (CF600E)
 - 总结一下

3 题目推荐

罗马游戏 (BZOJ1455)

一开始有 $n(n \leq 10^6)$ 个元素，每个元素权值为 a_i ，各自构成一个集合。

你需要执行如下的操作：

- 合并 i, j 所在的集合。
- 删除 i
- 询问 i 所在集合中的最小权值。

操作个数 $m \leq 10^5$ 。

- 很显然我们需要用并查集和某一种可以维护集合最小值和合并集合的数据结构完成这道题。

- 很显然我们需要用并查集和某一种可以维护集合最小值和合并集合的数据结构完成这道题。
- 左偏树就可以维护集合最小值和合并集合。

- 很显然我们需要用并查集和某一种可以维护集合最小值和合并集合的数据结构完成这道题。
- 左偏树就可以维护集合最小值和合并集合。
- 左偏树的合并复杂度为 $O(\log \max\{s_1, s_2\})$

Outline

简介

我是谁

数据结构是什么

正文

并查集

队列

栈

线段树

其他数据结构

题目推荐

- 很显然我们需要用并查集和某一种可以维护集合最小值和合并集合的数据结构完成这道题。
- 左偏树就可以维护集合最小值和合并集合。
- 左偏树的合并复杂度为 $O(\log \max\{s_1, s_2\})$
- 因此算法的复杂度为 $O(m \log n)$ 。

Lomsat gelral (CF600E)

给定一 $n(n \leq 10^5)$ 个点的树，点带颜色，求每个子树的支配颜色数量之和。

一个颜色是一个子树的支配颜色，当且仅当不存在其他的颜色在这个子树内的出现次数大于它。

Outline

简介

我是谁

数据结构是什么

正文

并查集

队列

栈

线段树

其他数据结构

题目推荐

- 这里需要用到一个叫做“树上启发式合并”的算法。

- 这里需要用到一个叫做“树上启发式合并”的算法。
- 这里的“启发式合并”和传统意义上的“启发式合并”不一样。

- 这里需要用到一个叫做“树上启发式合并”的算法。
- 这里的“启发式合并”和传统意义上的“启发式合并”不一样。
- 这里的答案和“玄学”一样，具有结合律，但是没有逆。因此我们只能累加答案，不能用前缀和等方法求答案。

- 这里需要用到一个叫做“树上启发式合并”的算法。
- 这里的“启发式合并”和传统意义上的“启发式合并”不一样。
- 这里的答案和“玄学”一样，具有结合律，但是没有逆。因此我们只能累加答案，不能用前缀和等方法求答案。
- 另外，这个答案也不适合用线段树去维护。

- 这里需要用到一个叫做“树上启发式合并”的算法。
- 这里的“启发式合并”和传统意义上的“启发式合并”不一样。
- 这里的答案和“玄学”一样，具有结合律，但是没有逆。因此我们只能累加答案，不能用前缀和等方法求答案。
- 另外，这个答案也不适合用线段树去维护。
- 唯一的办法就是像莫队那样累加答案。

- 这里需要用到一个叫做“树上启发式合并”的算法。
- 这里的“启发式合并”和传统意义上的“启发式合并”不一样。
- 这里的答案和“玄学”一样，具有结合律，但是没有逆。因此我们只能累加答案，不能用前缀和等方法求答案。
- 另外，这个答案也不适合用线段树去维护。
- 唯一的办法就是像莫队那样累加答案。
- 方法是这样的：

树上启发式合并

对于一个子树问题，我们可以用类似莫队的算法求解。对于一个节点 x

- 先求 x 所有轻子树的答案。
- 从数据结构中删掉 x 所有轻子树的贡献。
- 求 x 的重子树的答案，并保留在数据结构中。
- 将 x 的所有轻子树的贡献添加到数据结构中。
- 获得 x 的答案并返回。

树上启发式合并

对于一个子树问题，我们可以用类似莫队的算法求解。对于一个节点 x

- 先求 x 所有轻子树的答案。
- 从数据结构中删掉 x 所有轻子树的贡献。
- 求 x 的重子树的答案，并保留在数据结构中。
- 将 x 的所有轻子树的贡献添加到数据结构中。
- 获得 x 的答案并返回。

树上启发式合并复杂度证明

每一个点的贡献会被插入并从数据结构中删除若干次。
该次数正比于这个点到根节点的路径上轻边的条数。
而轻边的条数是 $O(\log n)$ 的。

- 综上所述，解决这道题的复杂度为 $O(n \log n f(n))$ 的。其中 $f(n)$ 为使用的数据结构单次操作的复杂度。

- 综上所述，解决这道题的复杂度为 $O(n \log n f(n))$ 的。其中 $f(n)$ 为使用的数据结构单次操作的复杂度。
- 如果使用桶来维护答案，时间复杂度为 $O(n \log n)$ 。

总结一下:

总结一下：

- 在 OI 中还有许多数据结构。

总结一下：

- 在 OI 中还有许多数据结构。
- 比如最常用的有 Treap, Splay 等平衡树，以及 ST 表，主席树，二叉搜索树等数据结构。

总结一下：

- 在 OI 中还有许多数据结构。
- 比如最常用的有 Treap, Splay 等平衡树，以及 ST 表，主席树，二叉搜索树等数据结构。
- 一道题使用哪一种数据结构去解决其实是次要的，重要的是搞清楚这道题的需求，而选择最合适的数据结构的过程。

总结一下：

- 在 OI 中还有许多数据结构。
- 比如最常用的有 Treap, Splay 等平衡树，以及 ST 表，主席树，二叉搜索树等数据结构。
- 一道题使用哪一种数据结构去解决其实是次要的，重要的是搞清楚这道题的需求，而选择最合适的数据结构的过程。
- 只有能熟练分析应该使用的数据结构，才能避免在做题时大炮打蚊子，大材小用，浪费时间。

Outline

简介

我是谁

数据结构是什么

正文

并查集

队列

栈

线段树

其他数据结构

题目推荐

- BZOJ1010 玩具装箱
- BZOJ3675 序列分割
- IOI2018 狼人
- CF786B Legacy
- HDU6430 Tea Tree
- BZOJ2809 dispatching
- luogu P3273 棘手的操作
- BZOJ4977 跳伞求生
- UOJ455 雪灾与外卖
- ICPC WF2018 征服世界
- NOI2017 蔬菜
- luogu P2605 基站选址
- CF452F Permutation
- luogu P4747 Intrinsic Interval
- BZOJ1098 办公楼
- UOJ356 Port Facility
- CF750D Tree Requests
- luogu P4647 [IOI2007] sails 船帆
- luogu P5283 异或粽子